

Cursor AI Cheatsheet (2025)

An advanced code editor with integrated AI capabilities, designed to enhance developer productivity through intelligent assistance.

 **by Mark Kashaf**

```
1 def greet(greet:
2     print("name!"):
3     print(f"Hi, {greet}")
4
5 if __name__ == '__main__':
6     user_name = "hey Alice!"
7     greet(user_name)
8
9 Thank you!
10 You're welcome!
```

AI ASSISTANT



How can I define a function in Python?

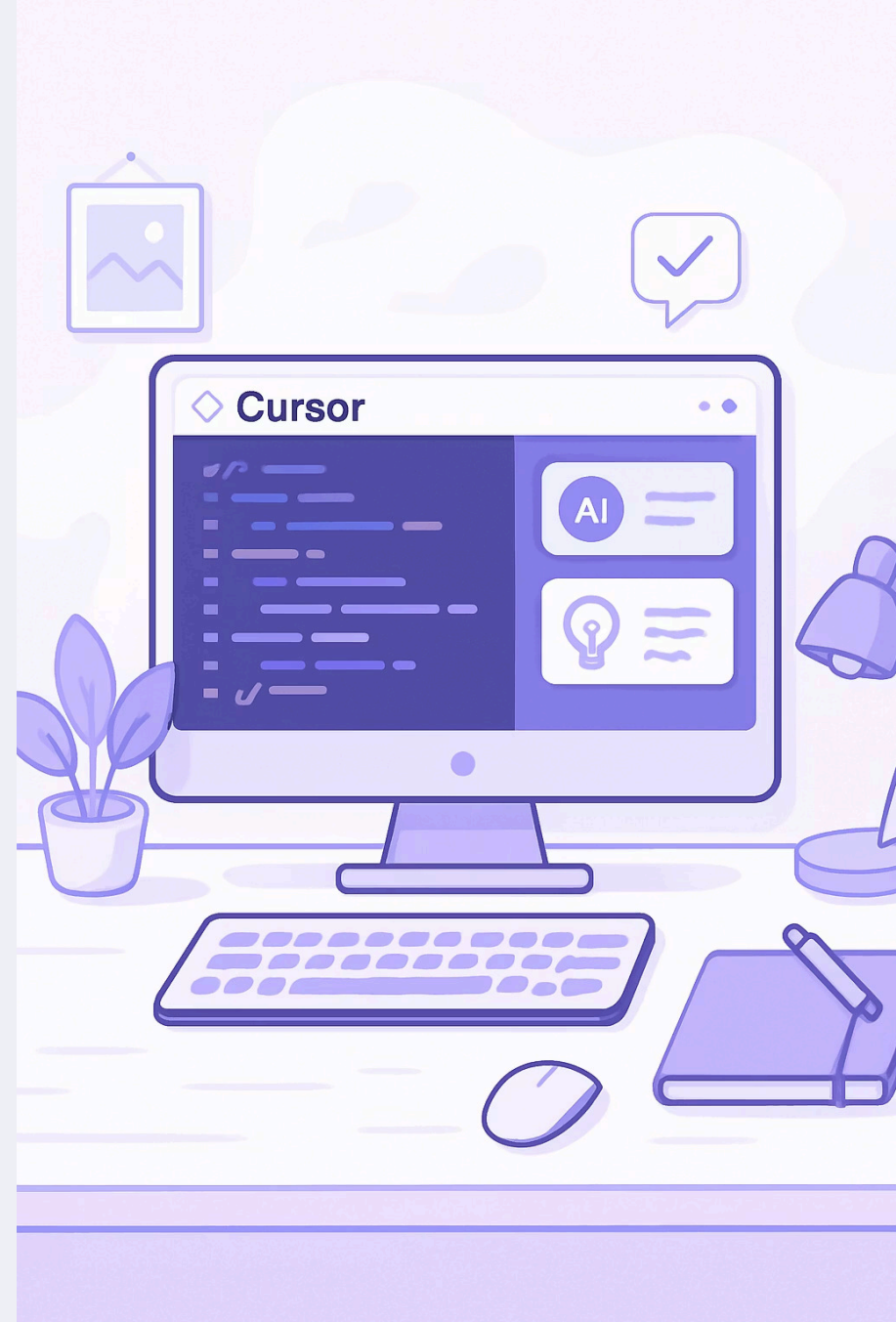
To define a function in Python, use the `def` keyword. Here's an example.

```
def greet(name):
    print(f"Hi, {name}")
```

Thank you!

Tool Overview

Cursor AI is an advanced code editor with integrated AI capabilities, designed to enhance developer productivity through intelligent assistance. It features a unified Agent Mode interface, support for multiple AI models including GPT-4.1, and a range of tools for autonomous code exploration and modification.



Core Technology



AI Engine

Multiple model support including GPT-4.1, Gemini 2.5, Grok 3, and Claude models



Development Environment

Browser-based code editor with integrated AI capabilities



Agent Mode

Unified interface for handling complex coding tasks with minimal guidance



Terminal Integration

Built-in terminal with command execution capabilities



Web Search

Integrated web browsing for documentation and solutions

Key Features at a Glance



Agent Mode

Unified autonomous interface



Web Search Integration

Documentation access



Rules Generation

Management for consistent behavior



Chat Tabs

Parallel conversations



Multiple AI Models

Support for GPT-4.1 and other advanced models



Terminal Command Execution

Improved control



Project Structure Understanding

Better context



Diff View

Code review and changes

Key Strengths



Unified Interface

Agent Mode provides a consistent approach to complex tasks



Autonomous Exploration

Independently explores codebases and identifies relevant files



Web Research

Access external resources to solve complex problems



Project Understanding

Option to include project structure in context



Model Flexibility

Support for multiple AI models to suit different needs and budgets



Terminal Integration

Execute commands directly from the AI interface



Rules System

Generate and manage rules for consistent behavior



Parallel Workflows

Chat tabs for managing multiple conversations

Limitations



Context Window

Approximately 120,000 tokens limit for GPT-4.1



Rule Sensitivity

GPT-4.1 follows defined rules more strictly than other models



Model Benefits

Some users report not seeing significant advantages of newer models in agent mode



Beta Features

Some capabilities still in development (e.g., project structure in context)



Instruction Conflicts

Potential for confusion with redundant or conflicting instructions



Learning Curve

Requires adjustment to effectively leverage all capabilities

Communication & Interface Guide

Effective communication with Cursor AI requires understanding its interface and capabilities. The following sections outline how to best interact with the system for optimal results.

Agent Mode

Agent Mode is the default and most autonomous mode in Cursor, designed to handle complex coding tasks with minimal guidance:

Key Capabilities:

- Independently explores codebases
- Identifies relevant files for modifications
- Uses all available tools (search, edit, terminal)
- Builds comprehensive understanding of project structure
- Breaks complex tasks into manageable steps
- Creates checkpoints before making changes

Best Practices:

- Be specific about what you want to accomplish
- No need to specify how the agent should approach the task
- Write clear and non-conflicting rules
- Avoid redundancy in instructions
- Review proposed changes before confirming

Example Agent Mode Prompts

Dark Mode Implementation

"Add dark mode support to this React application. The app should detect system preferences and also provide a toggle for users to switch between light and dark themes."

API Response Caching

"Implement API response caching for all fetch requests in this application to improve performance and reduce unnecessary network calls."

Authentication Bug Fix

"Fix the bug in the user authentication flow where users are not being redirected to the dashboard after successful login."

Chat Tabs and Modes

Cursor provides multiple chat tabs and interaction modes:

Chat Tabs:

- Create multiple tabs for parallel conversations
- Orange dot indicator for tabs awaiting input
- Access chat history through command palette
- Sound notification option when chat is finished

Interaction Modes:

- **Agent Mode:** Full autonomous assistance
- **Ask Mode:** Question-answering without code changes
- **Manual Mode:** Direct control over changes
- **Custom Modes:** User-defined interaction patterns

Rules Generation

Rules help guide the agent's behavior consistently:

How to Use:

1. Use the `/Generate Cursor Rules` command in chat
2. Review and edit the generated rules
3. Apply rules with path patterns for automatic application
4. Configure global ignore patterns via user-level settings

// General coding style

- Use TypeScript for all new files
- Follow the existing project structure
- Add comprehensive comments for complex logic
- Write unit tests for new functionality

// Path patterns

- `src/components/**/*`.tsx: Follow component structure with props interface
- `src/utils/**/*`.ts: Create pure functions with JSDoc comments
- `src/api/**/*`.ts: Use try/catch for all API calls

Workflow Optimization

Optimizing your workflow with Cursor AI involves understanding the systematic approach of Agent Mode, selecting appropriate models, and leveraging collaboration features.

Agent Mode Workflow

Understand Request

Analyzes request and codebase context

Explore Codebase

Searches through files, documentation, and web

Plan Changes

Breaks down task into smaller steps

Execute Changes

Makes necessary code modifications

Verify Results

Confirms changes and fixes any issues

Task Complete

Summarizes the changes made

Model Selection Strategy

Choose the appropriate AI model based on your needs:

Model	Best For
GPT-4.1 (Standard)	Complex coding tasks requiring deep understanding
GPT-4.1-mini	Balanced performance and cost efficiency
GPT-4.1-nano	Simple tasks with budget constraints
Gemini 2.5 Pro	Alternative reasoning approaches
Grok 3	Creative problem-solving
Claude models	Detailed explanations and reasoning

AI MODELS AND USE CASES

AI Model	Best Use Case
 GPT-4	Text generation
 Gemini	Question answering
 Claude	Summarization
 Mistral	Code completion

Effective Collaboration



Organize with Chat Tabs

Use chat tabs to organize different aspects of a project



Leverage Chat History

Reference previous conversations for context



Review Changes

Use the built-in diff view to examine proposed modifications



Create Checkpoints

Save state before making significant changes



Project Structure

Use project structure understanding for better context

Terminal Integration

Leverage the integrated terminal for command execution:

- Run commands directly from the AI interface
- Edit commands before execution
- Skip commands or move them to background
- View exit codes for better error handling
- Run commands in background for long-running processes

Performance Optimization

Maximizing the effectiveness of Cursor AI requires attention to model efficiency, code quality, project organization, and debugging strategies.

Model Efficiency



Select appropriate models

Choose models based on task complexity



Optimize prompts

Be specific and concise to reduce token usage



Use OpenRouter API

Access over 300 AI models through a single interface



Monitor usage

Track token consumption and costs



Configure budget alerts

Set usage thresholds to avoid unexpected charges

Code Quality



Request code reviews

Ask the agent to review and improve code



Implement best practices

Request adherence to coding standards



Optimize algorithms

Ask for performance improvements



Reduce technical debt

Request refactoring of problematic code



Improve error handling

Ask for comprehensive error management

Project Organization



Include project structure

Enable project structure in context



Configure global ignore patterns

Exclude unnecessary files



Use auto-attached rules

Apply consistent patterns across files



Organize with chat tabs

Separate concerns into different conversations



Leverage MCP support

Use multi-context processing for complex tasks

Debugging Strategies



Terminal integration

Run tests and debugging commands



Linting error fixing

Automatically address code quality issues



Web search

Research solutions to complex problems



Diff view

Review changes before applying them

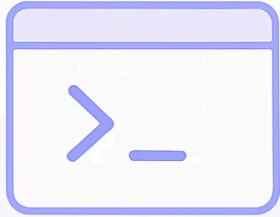


Checkpoint system

Revert to previous states if needed



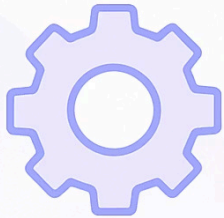
Web Search



Terminal



File System



External Tools

Integration Capabilities

Cursor AI offers robust integration with various systems and tools to enhance your development workflow.

Web Search Integration



Documentation access

Search for library and framework documentation



Problem-solving

Find solutions to coding challenges



Best practices

Research industry standards and patterns



Contextual searches

Integrate search results with codebase understanding



External resources

Access tutorials, examples, and community discussions

Terminal Command Execution



Package management

Install and update dependencies



Build processes

Compile and bundle applications



Testing

Run test suites and view results



Deployment

Execute deployment scripts



Database operations

Manage database migrations and queries

File System Integration



Code modification

Edit existing files with context awareness



File creation

Generate new files based on requirements



Directory navigation

Explore project structure



Checkpoint system

Create safe points before making changes



Diff view

Compare changes before applying them

External Tools



OpenRouter API

Access over 300 AI models



Version control

Integrate with Git workflows



CI/CD pipelines

Trigger and monitor build processes



Code analysis

Integrate with linting and static analysis tools



Project management

Connect with issue tracking systems

PROJECT TYPES



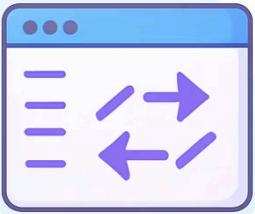
WEB
DEVELOPMENT



API
INTEGRATION



BUG
FIXING



CODE
REFACTORING



DOCUMENTATION

Project Type Optimization

Cursor AI excels at different types of projects, with specific optimization strategies for each.

Best For

Web Application Development

- Frontend frameworks (React, Vue, Angular)
- Backend systems (Node.js, Python, Java)
- Full-stack applications
- Progressive web apps
- Single-page applications

API Integration and Implementation

- RESTful API development
- GraphQL schema design
- API client implementation
- Webhook processing
- Authentication flows

Bug Fixing and Debugging

- Error identification
- Root cause analysis
- Solution implementation
- Regression testing
- Performance bottlenecks

More Project Types

Code Refactoring

- Technical debt reduction
- Architecture improvements
- Pattern implementation
- Code modernization
- Performance optimization

Documentation Generation

- Code comments
- API documentation
- README files
- User guides
- Technical specifications

Troubleshooting Guide

Common Issues

- Context Window Limitations
- Model Selection Problems
- Rule Conflicts
- Terminal Command Issues
- Web Search Integration Problems

Debugging Workflow

1. Identify the issue
2. Gather information
3. Isolate the cause
4. Develop a solution
5. Review changes
6. Test thoroughly
7. Document the solution

Support Resources

- Documentation: docs.cursor.com
- Community Forum: forum.cursor.com
- GitHub Repository: github.com/cursor-ai
- Twitter: [@cursor_ai](https://twitter.com/cursor_ai)
- Discord: discord.gg/cursor

Keyboard Shortcuts

- **Ctrl+Shift+P / Cmd+Shift+P**: Open command palette
- **Ctrl+/ / Cmd+/:** Toggle comment
- **Ctrl+F / Cmd+F**: Find in file
- **Ctrl+Shift+F / Cmd+Shift+F**: Find in project
- **Ctrl+ / Cmd+:** Toggle terminal
- **Ctrl+Tab / Cmd+Tab**: Switch between chat tabs